Contents

ms-fidelizacion — Justificación del servicio y cobertura de requisitos **1**
1. Misión del servicio . . . 1
2. Requisitos funcionales que cubre . . . 1
3. Requisitos no funcionales que cubre . . . 1
4. Requisitos de seguridad que cubre (mapeo STRIDE) . . . 2
5. Stack tecnológico y por qué cada tecnología . . . 3
6. Delimitación: qué NO hace ms-fidelizacion (IE9/IE10) . . . 4
7. Diagramas que respaldan esta justificación . . . 4

ms-fidelizacion — Justificación del servicio y cobertura de requisitos

Caso caso06 — **ViajaMejor** (Reservas de viajes) · EP01 JVY0101

Este documento justifica la existencia de **ms-fidelizacion** como microservicio independiente: qué requisitos del negocio cubre (funcionales, no funcionales y de seguridad), por qué está delimitado así (SRP), y qué tecnología AWS se usa para cada responsabilidad y **por qué**. Los diagramas que respaldan esta justificación están en `docs/diagramas/`.

---

1. Misión del servicio

ms-fidelizacion orquesta las operaciones de negocio del dominio, garantiza la consistencia de los datos que le corresponden y publica los eventos que desencadenan el resto del proceso del caso caso06 (ViajaMejor).

> Es el único servicio que puede dejar al negocio en un estado inconsistente si pierde datos: por eso toda escritura se persiste antes de confirmar y toda comunicación posterior sale por colas de mensajes (EDA), nunca en línea con el usuario esperando.

---

2. Requisitos funcionales que cubre

RF	Requisito (de `00_PresentacionEmpresa.md`)	Qué hace ms-fidelizacion al respecto	Evidencia
RF-07	Acumular y canjear puntos de fidelización	Orquesta la operación del dominio: valida, persiste, publica el evento de dominio y expone el estado	diagrama de secuencia (ciclo de vida de la operación)

Por qué estos RF justifican un servicio aparte: Es el único servicio que puede dejar al negocio en un estado inconsistente si pierde datos: por eso toda escritura se persiste antes de confirmar y toda comunicación posterior sale por colas de mensajes (EDA), nunca en línea con el usuario esperando.

---

3. Requisitos no funcionales que cubre

RNF	Criterio	Cómo lo cumple ms-fidelizacion	Decisión técnica
RNF-01 (Escalabilidad)	Soportar picos de búsqueda en temporada alta escalando el buscador de forma independiente	Auto scaling independiente de este servicio (2→10 tareas Fargate según carga)	ECS Fargate + alarmas de CloudWatch: solo este componente escala en el pico
RNF-02 (Disponibilidad)	La búsqueda y disponibilidad deben operar aunque el servicio de pagos o un proveedor externo falle	Aislamiento por eventos: este servicio sigue operando aunque fallen los vecinos	Comunicación asíncrona (SQS/EventBridge) + multi-AZ
RNF-06 (Consistencia)	Las reservas deben sobrevivir a fallas; el pago debe ser atómico con la confirmación de la reserva	Escritura persistida antes de confirmar; eventos por cola con DLQ ante fallas	Aurora transaccional + SQS persistido + idempotencia
RNF-05 (Rendimiento)	Resultados de búsqueda de disponibilidad en menos de 3 segundos	Respuestas dentro del umbral exigido por el caso (ver alarmas p95)	Caché/bajo acoplamiento + CloudWatch con alarma de latencia

Justificación SRP (IE9): ms-fidelizacion tiene **una sola razón de cambio**: las reglas del proceso de negocio: estados, validaciones y flujo de la operación. Si mañana cambia esa regla, **ningún otro servicio se modifica**.

---

4. Requisitos de seguridad que cubre (mapeo STRIDE)

Amenaza	Escenario en este servicio	Contramedida
Spoofing	Operar en nombre de otro usuario	JWT validado en API Gateway; el recurso siempre se asocia al identity del token
Tampering	Alterar una operación en curso	Validación de negocio en el servicio + escritura transaccional en la BD propia (JPA)
Repudiation	Negar una operación realizada	Persistencia inmediata del estado + evento de dominio + CloudTrail
Information disclosure	Consultar operaciones ajenas	Autorización por recurso (owner) y cifrado at-rest con KMS
Denial of service	Saturar el servicio en hora punta	Auto scaling (Fargate) + throttling en API Gateway + degradación con Circuit Breaker

Amenaza	Escenario en este servicio	Contramedida
Elevation of privilege	Ejecutar operaciones de otro rol	Roles del JWT verificados por endpoint; credenciales de BD vía Secrets Manager, no en código

---

5. Stack tecnológico y por qué cada tecnología

5.1 Stack de la aplicación

Tecnología	Para qué se usa en ms-fidelizacion
Java 21 + Spring Boot 3.3	Framework estándar de la asignatura: implementa la API REST, la lógica de negocio y el acceso a datos del servicio
Spring Data JPA	Persistencia de las entidades del dominio en la base de datos propia (repositorios por entidad)
Bean Validation	Validación de los payloads de entrada antes de procesar (jakarta.validation)
springdoc-openapi	Documentación viva del contrato REST (Swagger UI / ReDoc) para consumidores y equipo
Docker + Docker Compose	Empaquetado reproducible; la misma imagen corre en local y en ECS Fargate
JUnit 5 + Mockito + MockMvc	Pruebas unitarias y de contrato HTTP (cobertura 100 % LINE con JaCoCo)
Cucumber (BDD)	Escenarios en español alineados a los endpoints, ejecutados contra el servidor real

5.2 Stack AWS y justificación de cada servicio

Servicio AWS	Rol en ms-fidelizacion	Por qué se eligió
ECS Fargate	Runtime del servicio con auto scaling 2→10 tareas	Escala SOLO este servicio en los picos del negocio (RNF de escalabilidad)
Amazon Aurora Serverless	BD transaccional propia del dominio	Garantiza atomicidad de las operaciones del caso (RNF de consistencia)
Amazon SQS (+ DLQ)	Cola persistida de eventos del dominio	Ninguna operación se pierde ante fallas; DLQ para reprocesar (RNF de integridad)
Amazon EventBridge	Bus de eventos hacia notificaciones y analítica	Desacople total: añadir consumidores no modifica este servicio (IE10)
Amazon API Gateway	Entrada única con JWT y throttling	Protege y rate-limita las operaciones en hora punta
CloudWatch + X-Ray	Alarmas de latencia y trazas distribuidas	Monitoreo del flujo crítico de punta a punta (IE8)

5.3 Patrones aplicados (IE5)

Patrón	Dónde
API Gateway	Entrada única con JWT y throttling
Circuit Breaker	Resilience4j en las llamadas síncronas a vecinos
Event-Driven Architecture	Publica el evento de dominio por SQS/EventBridge para desacoplar el paso siguiente
Saga por coreografía	Cada paso confirma y publica; la falla dispara compensación

---

6. Delimitación: qué NO hace ms-fidelizacion (IE9/IE10)

No hace	Lo hace	Por qué
usuarios	ms-usuarios	razones de cambio distintas: la autenticación se centraliza aquí, pero el negocio de cada dominio queda en su servicio
búsqueda	ms-busqueda	razones de cambio distintas: el catálogo consulta y publica; las operaciones de negocio las orquesta el servicio transaccional
pagos	ms-pagos	razones de cambio distintas: el dinero se procesa aquí, pero la operación que lo origina vive en el servicio central
integraciones	ms-integraciones	razones de cambio distintas: el seguimiento vive aquí, pero la operación que lo origina vive en el servicio central

---

7. Diagramas que respaldan esta justificación

```
docs/diagramas/
├── c4/
│   ├── C4-1-Contexto     el servicio, sus actores y sus vecinos
│   ├── C4-2-Contenedor   la API, la BD propia y los componentes del dominio
│   └── C4-3-Componentes  validador/service, clientes, publicador, repos
├── secuencia/
│   └── Secuencia-Punto   ciclo de vida de la operación
└── infraestructura/
    └── Infra-AWS         despliegue solo de este servicio, con iconos oficiales AWS
```